The Quantum Fourier Transform

From sound waves to quantum circuits — explained step by step

CUDA-Q Quantum Algorithm Benchmarking Suite

For first-year engineering students

All quantum results in these slides were computed with NVIDIA CUDA-Q and checked against NumPy.

What you will learn today

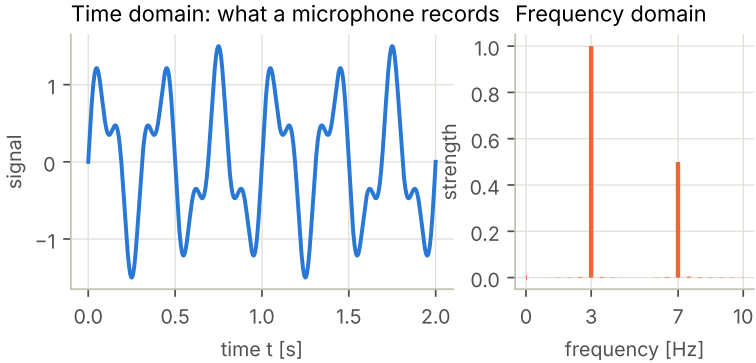
1. Why engineers love the **Fourier transform**
2. The math toolkit: **complex numbers**, roots of unity, binary fractions
3. **Qubits** in ten minutes
4. The **Quantum Fourier Transform (QFT)**: definition and small examples
5. How to **build the QFT circuit** from simple gates
6. Examples: **periodic signals**, peaks and leakage (with real simulation results)
7. Where the QFT is used, its limits, and how to run it in **CUDA-Q**

You only need

- school algebra (powers, sums)
- vectors and matrices (multiplying a matrix by a vector)
- a little trigonometry (sin, cos)

Everything else is explained on the way.

Every signal is a sum of waves



The signal on the left looks complicated, but it is just

$$s(t) = \underbrace{\sin(2\pi \cdot 3\,t)}_{\text{3 Hz wave}} + \underbrace{0.5\,\sin(2\pi \cdot 7\,t)}_{\text{7 Hz wave}}.$$

The **Fourier transform** finds these ingredients: **which frequencies, and how strong**.

Fourier analysis is everywhere in engineering

- **Audio**: MP3 and equalisers work on frequencies
- **Images**: JPEG stores frequency coefficients
- **Telecom**: Wi-Fi, 4G/5G (OFDM) send data on many frequencies at once
- **Medicine**: MRI images are reconstructed with Fourier transforms
- **Mechanics**: vibration analysis finds the resonance frequencies of bridges, engines, wings

The common idea

Change the point of view:

time / position $\longrightarrow$ frequency

Some questions are hard in one view and easy in the other — for example **"what is the period of this signal?"**

The quantum computer uses exactly this trick to find hidden periods (Shor's factoring algorithm).

The Discrete Fourier Transform (DFT)

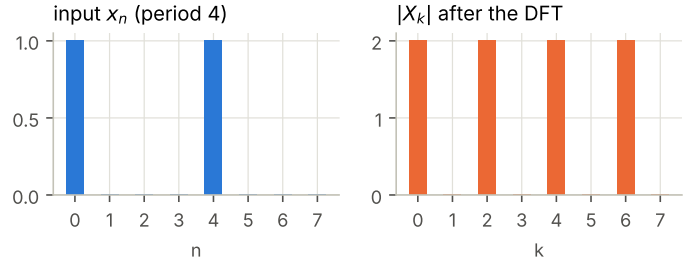
A computer stores a signal as M samples $x_0, x_1, \ldots, x_{M-1}$. The DFT turns them into M frequency coefficients:

Discrete Fourier Transform

$$X_k = \sum_{n=0}^{M-1} x_n \, e^{-2\pi i \, kn/M}, \qquad k = 0, 1, \ldots, M-1.$$

- k = frequency index: "how many full oscillations fit into the M samples"
- $e^{-2\pi ikn/M}$ is a wave written with complex numbers (next part explains why)
- $|X_k|$ = how strongly frequency k is present in the signal

Example: the DFT finds the period



A spike every $r = 4$ samples. Only $n = 0, 4$ contribute and $e^{-2\pi i \cdot 4k/8} = (-1)^k$, so

$$X_k = 1 + (-1)^k = \begin{cases} 2 & k \text{ even} \\ 0 & k \text{ odd} \end{cases}$$

Remember this pattern

Period r in the input $\Rightarrow$ peaks at multiples of M/r in the output (here $8/4 = 2$: $k = 0, 2, 4, 6$).

How expensive is a Fourier transform?

- Direct formula: M outputs $\times$ M terms $= M^2$ multiplications
- **Fast Fourier Transform (FFT)**, 1965: only about $M \log_2 M$

M	M^2	$M \log_2 M$
$1\,024$	10^6	10^4
10^6	10^{12}	2×10^7

The quantum idea (preview)

n qubits hold $M = 2^n$ numbers at once.

The QFT transforms all of them with only about $n^2/2$ simple operations.

For $n = 30$: $M \approx 10^9$ numbers, but only ≈ 480 gates.

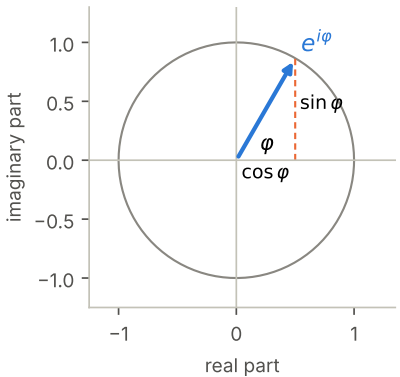
But: we cannot simply read out all M results — we will see why and how algorithms work around it.

Complex numbers and Euler's formula

- A complex number $z = a + ib$ with $i^2 = -1$ is a point in the plane: real part a, imaginary part b.
- Length (magnitude): $|z| = \sqrt{a^2 + b^2}$
- **Euler's formula** — the most useful identity of this course:

$$e^{i\varphi} = \cos\varphi + i\,\sin\varphi$$

$e^{i\varphi}$ is the point on the **unit circle** at angle φ. Every complex number is $z = |z|\,e^{i\varphi}$: a length and a direction ("phase").



Multiplying phases = adding angles

$$e^{i\alpha} \cdot e^{i\beta} = e^{i(\alpha+\beta)}$$

Multiplying by $e^{i\beta}$ **rotates** a point by the angle β.

Values to remember

$$e^{i\cdot 0} = 1 \qquad e^{i\pi/2} = i$$
$$e^{i\pi} = -1 \qquad e^{i3\pi/2} = -i$$
$$e^{2\pi i} = 1$$

The key fact for the QFT

A full turn changes nothing:

$$e^{2\pi i\, k} = 1 \quad \text{for every integer } k.$$

So in $e^{2\pi i\,(\text{number})}$ only the **fractional part** of the number matters:

$$e^{2\pi i\cdot 2.25} = e^{2\pi i\cdot 0.25} = i.$$

Roots of unity: equally spaced points on the circle

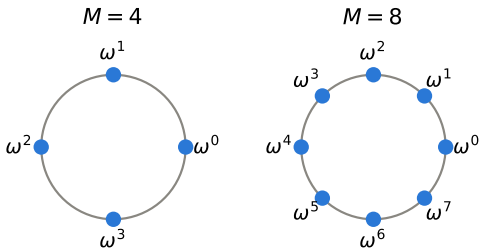
Define

$$\omega = e^{2\pi i/M}.$$

Its powers $\omega^0, \omega^1, \ldots, \omega^{M-1}$ are M equally spaced points on the unit circle, and $\omega^M = 1$.

- $M = 2$: $\omega = -1$
- $M = 4$: $\omega = i$
- $M = 8$: $\omega = e^{i\pi/4}$ (45° steps)

The QFT on n qubits uses $M = 2^n$.



ω^k = point at angle $k \cdot 360°/M$

Why peaks appear: arrows that cancel

Sum of roots of unity

$$\sum_{j=0}^{M-1} \omega^{jk} = \begin{cases} M & \text{if } k \text{ is a multiple of } M \\ 0 & \text{otherwise} \end{cases}$$

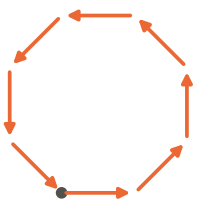
Proof (geometric series): for $\omega^k \neq 1$,
$\sum_j (\omega^k)^j = \dfrac{1-\omega^{kM}}{1-\omega^k} = \dfrac{1-1}{1-\omega^k} = 0.$

Adding arrows head-to-tail: if they all point the same way they add up; if they are spread evenly around the circle they **cancel**.

k = 0: all arrows aligned, sum = 8



k = 1: arrows go around, sum = 0



$M = 8$: the arrows ω^{jk}, $j = 0, \ldots, 7$, added head-to-tail

Binary numbers and binary fractions

Integers with n bits (we write the most significant bit first):

$$x = x_1 x_2 \dots x_n = x_1 2^{n-1} + \dots + x_n 2^0$$

Example: $5 = 101_2 = 4 + 0 + 1$.

Binary fractions:

$$0.x_1 x_2 \dots x_m = \frac{x_1}{2} + \frac{x_2}{4} + \dots + \frac{x_m}{2^m}$$

binary	decimal
0.1_2	$1/2 = 0.5$
0.01_2	$1/4 = 0.25$
0.11_2	$3/4 = 0.75$
0.101_2	$5/8 = 0.625$

Shifting the point

Dividing by 2 moves the binary point one place left: $5/2 = 10.1_2$, $5/4 = 1.01_2$, $5/8 = 0.101_2$.

Bit vs. qubit

A classical **bit** is 0 or 1.

A **qubit** is a unit vector of two complex numbers:

$$|0\rangle = \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \quad |1\rangle = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$$

$$|\psi\rangle = \alpha\,|0\rangle + \beta\,|1\rangle = \begin{pmatrix} \alpha \\ \beta \end{pmatrix}$$

with $|\alpha|^2 + |\beta|^2 = 1$. α, β are called **amplitudes**.

Superposition

$$|+\rangle = \tfrac{1}{\sqrt{2}}\big(\,|0\rangle + |1\rangle\,\big)$$

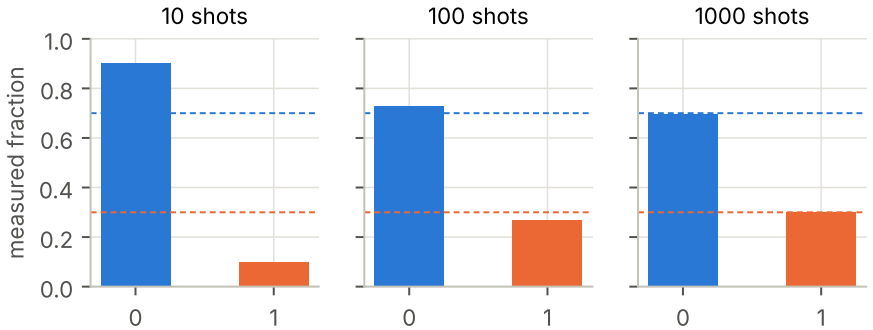
is "0 and 1 at the same time" — more precisely: a vector with two non-zero components.

The notation $|\cdot\rangle$ ("ket") just marks a column vector.

Measurement: amplitudes become probabilities

Born rule

Measuring $\alpha|0\rangle + \beta|1\rangle$ gives 0 with probability $|\alpha|^2$ and 1 with probability $|\beta|^2$. Afterwards the qubit *is* in the measured state.



A qubit with $|\alpha|^2 = 0.7$ measured 10, 100 and 1000 times (CUDA-Q). One **shot** gives one bit; the probabilities only appear after many shots (dashed lines: 0.7 and 0.3).

The phase: invisible, but it matters

$$|+\rangle = \tfrac{1}{\sqrt{2}}(|0\rangle + |1\rangle), \qquad |-\rangle = \tfrac{1}{\sqrt{2}}(|0\rangle - |1\rangle)$$

Both give 50 % / 50 % when measured — the minus sign (a **phase** $e^{i\pi}$) is invisible.

But after one more gate (the Hadamard H, next slide):

$$H\,|+\rangle = |0\rangle\,, \qquad H\,|-\rangle = |1\rangle$$

Now the phase decides the result!

Phase states

$$\tfrac{1}{\sqrt{2}}\big(\,|0\rangle + e^{i\varphi}\,|1\rangle\,\big)$$

All have 50/50 probabilities; they differ only in the angle φ — think of a **clock hand**.

The QFT writes information into exactly these angles.

Many qubits: 2^n amplitudes

- n qubits have 2^n basis states $|x_1 x_2 \ldots x_n\rangle$, one for every n-bit number x:

$$|\psi\rangle = \sum_{x=0}^{2^n-1} \alpha_x |x\rangle, \qquad \sum_x |\alpha_x|^2 = 1.$$

- Two qubits: $|\psi\rangle = \alpha_0 |00\rangle + \alpha_1 |01\rangle + \alpha_2 |10\rangle + \alpha_3 |11\rangle$
- Independent qubits combine with the **tensor product**:
  $(a|0\rangle + b|1\rangle) \otimes (c|0\rangle + d|1\rangle) = ac|00\rangle + ad|01\rangle + bc|10\rangle + bd|11\rangle$
- **Convention** in these slides (and in the CUDA-Q notebooks): qubit 1 (top wire) is the **most significant bit** x_1.

Why simulation is hard

A classical computer must store all 2^n complex numbers: 16 bytes each, so $n = 30$ needs 17 GB and $n = 50$ needs 18 petabytes.

Single-qubit gates are 2×2 matrices

NOT gate X

$$X = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$$

$X|0\rangle = |1\rangle$, $X|1\rangle = |0\rangle$

Hadamard H

$$H = \frac{1}{\sqrt{2}} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}$$

$H|0\rangle = |+\rangle$, $H|1\rangle = |-\rangle$

Phase gate R_k

$$R_k = \begin{pmatrix} 1 & 0 \\ 0 & e^{2\pi i/2^k} \end{pmatrix}$$

turns the $|1\rangle$ part by $360°/2^k$

- Compact form of the Hadamard: $H|b\rangle = \frac{1}{\sqrt{2}}(|0\rangle + (-1)^b|1\rangle)$ for $b \in \{0,1\}$
- $R_1 = Z$ (180°), $R_2 = S$ (90°), $R_3 = T$ (45°), R_4: 22.5°, ...
- Gates are **unitary**: they keep the vector length 1 and can always be undone.

Controlled gates

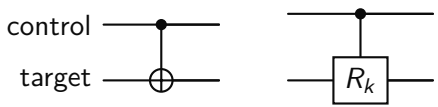
A **controlled-U** gate applies U to the target only if the control qubit is $|1\rangle$:

$$|0\rangle\,|\psi\rangle \mapsto |0\rangle\,|\psi\rangle\,, \qquad |1\rangle\,|\psi\rangle \mapsto |1\rangle\ U\,|\psi\rangle$$

- controlled-X $=$ **CNOT**
- **controlled-R_k**: multiplies only $|11\rangle$ by $e^{2\pi i/2^k}$:

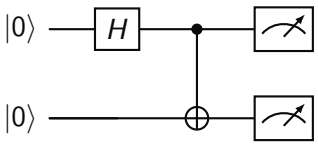
$$\mathrm{diag}\left(1, 1, 1, e^{2\pi i/2^k}\right)$$

Control and target can be swapped — it is symmetric!



CNOT (left) and controlled-R_k (right). A dot marks the control.

Reading a quantum circuit



- Each horizontal line is a qubit; time runs **left to right**.
- Step 1: H on the top qubit: $\frac{1}{\sqrt{2}}(|00\rangle + |10\rangle)$
- Step 2: CNOT flips the bottom qubit when the top one is 1:

$$\frac{1}{\sqrt{2}}\left(\,|00\rangle + |11\rangle\,\right)$$

- Step 3: measure: 00 or 11, 50 % each — never 01 or 10 (the qubits are **entangled**).

Definition of the QFT

For n qubits, $M = 2^n$ and $\omega = e^{2\pi i/M}$:

Quantum Fourier Transform

$$\mathrm{QFT}\ |x\rangle = \frac{1}{\sqrt{M}} \sum_{y=0}^{M-1} e^{2\pi i\, xy/M}\ |y\rangle = \frac{1}{\sqrt{M}} \sum_{y=0}^{M-1} \omega^{xy}\ |y\rangle$$

- The input basis state $|x\rangle$ becomes an equal superposition of *all* $|y\rangle$ ...
- ... where $|y\rangle$ carries the phase ω^{xy}: a wave with "frequency" x.
- The factor $1/\sqrt{M}$ keeps the total probability equal to 1: $M \times |1/\sqrt{M}|^2 = 1$.

The QFT is the DFT of the amplitudes

The QFT is linear, so a general state $|\psi\rangle = \sum_x \alpha_x |x\rangle$ is transformed term by term:

$$\mathrm{QFT}\,|\psi\rangle = \sum_y \beta_y |y\rangle \qquad \text{with} \qquad \beta_y = \frac{1}{\sqrt{M}} \sum_{x=0}^{M-1} \alpha_x \, \omega^{xy}.$$

Compare with the DFT

Same formula, but the sign in the exponent is $+$ and there is a factor $1/\sqrt{M}$ — just conventions. In NumPy: `np.fft.ifft(alpha, norm="ortho")`.

What is transformed?

Not a list of numbers in memory, but the **amplitudes** of the quantum state — all 2^n of them at once.

The QFT as a matrix

Column x of the matrix is the output for input $|x\rangle$; the entry in row y is $\omega^{xy}/\sqrt{M}$:

$$F_M = \frac{1}{\sqrt{M}} \begin{pmatrix} 1 & 1 & 1 & \cdots & 1 \\ 1 & \omega & \omega^2 & \cdots & \omega^{M-1} \\ 1 & \omega^2 & \omega^4 & \cdots & \omega^{2(M-1)} \\ \vdots & & & \ddots & \vdots \\ 1 & \omega^{M-1} & \omega^{2(M-1)} & \cdots & \omega^{(M-1)^2} \end{pmatrix}$$

- Row y is a wave that turns by $360^\circ \cdot y/M$ per column.
- Multiplying this $M \times M$ matrix by a vector costs M^2 operations classically — the quantum circuit will do it with about $n^2/2$ gates.

Smallest case: one qubit

$n = 1$, $M = 2$, $\omega = e^{2\pi i/2} = e^{i\pi} = -1$:

$$\mathrm{QFT}\,|0\rangle = \tfrac{1}{\sqrt{2}}(\omega^0\,|0\rangle + \omega^0\,|1\rangle\,) = \tfrac{1}{\sqrt{2}}(\,|0\rangle + |1\rangle\,)$$

$$\mathrm{QFT}\,|1\rangle = \tfrac{1}{\sqrt{2}}(\omega^0\,|0\rangle + \omega^1\,|1\rangle\,) = \tfrac{1}{\sqrt{2}}(\,|0\rangle - |1\rangle\,)$$

Result

$$F_2 = \tfrac{1}{\sqrt{2}}\begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} = H$$

The one-qubit QFT is simply the **Hadamard gate**.

Two qubits: $M = 4$, $\omega = i$

$$F_4 = \frac{1}{2}\begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & i & -1 & -i \\ 1 & -1 & 1 & -1 \\ 1 & -i & -1 & i \end{pmatrix}$$

Entry in row y, column x: $i^{xy}/2$.
(Use $i^2 = -1$, $i^3 = -i$, $i^4 = 1$.)

Check: columns are orthonormal

Columns $x = 1$ and $x = 2$ (conjugate the first):

$$\tfrac{1}{4}(1 + \overline{i} \cdot (-1) + (-1)(1) + \overline{(-i)}(-1))$$

$$= \tfrac{1}{4}(1 + i - 1 - i) = 0 \checkmark$$

Orthonormal columns = a unitary matrix = a valid quantum gate.

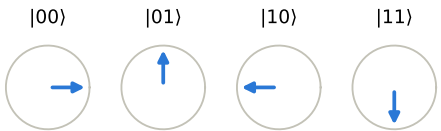
Worked example: QFT on $|01\rangle$ and $|10\rangle$

Read off column x of F_4:

$$\mathrm{QFT}\,|01\rangle = \tfrac{1}{2}\big(\,|00\rangle + i\,|01\rangle - |10\rangle - i\,|11\rangle\,\big) \qquad (x = 1)$$

$$\mathrm{QFT}\,|10\rangle = \tfrac{1}{2}\big(\,|00\rangle - |01\rangle + |10\rangle - |11\rangle\,\big) \qquad (x = 2)$$

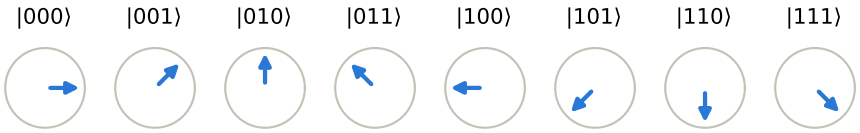
QFT|01⟩: arrow turns by 90 ° per step



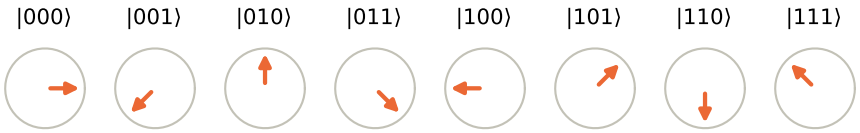
Each amplitude drawn as a clock hand (CUDA-Q result). For $x = 1$ the hand turns 90° from one $|y\rangle$ to the next; for $x = 2$ it turns 180°. **The input x sets the speed of rotation** — a frequency.

Three qubits: phase clocks

QFT|001⟩ ($x = 1$): arrow turns by 45 ° per step



QFT|101⟩ ($x = 5$): arrow turns by 5 × 45 ° per step



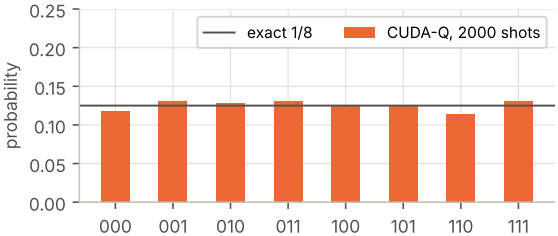
$M = 8$: the amplitude of $|y\rangle$ in $\mathrm{QFT}\,|x\rangle$ points at angle $360° \cdot xy/8$. All arrows have the same length $1/\sqrt{8}$ — only the **directions** encode x.

Checking the numbers with CUDA-Q: $\mathrm{QFT}\,|101\rangle$

$\lvert y\rangle$	y	Re	Im	angle	$\lvert\beta_y\rvert^2$
$\lvert 000\rangle$	0	$+0.3536$	$+0.0000$	$0°$	0.1250
$\lvert 001\rangle$	1	-0.2500	-0.2500	$225°$	0.1250
$\lvert 010\rangle$	2	$+0.0000$	$+0.3536$	$90°$	0.1250
$\lvert 011\rangle$	3	$+0.2500$	-0.2500	$315°$	0.1250
$\lvert 100\rangle$	4	-0.3536	$+0.0000$	$180°$	0.1250
$\lvert 101\rangle$	5	$+0.2500$	$+0.2500$	$45°$	0.1250
$\lvert 110\rangle$	6	$+0.0000$	-0.3536	$270°$	0.1250
$\lvert 111\rangle$	7	-0.2500	$+0.2500$	$135°$	0.1250

- Simulated with CUDA-Q (`cudaq.get_state`).
- Angle $= 225° \cdot y \bmod 360°$, exactly ω^{5y} with $\omega = e^{i\pi/4}$.
- $0.3536 = 1/\sqrt{8}$, $0.25 = \frac{1}{\sqrt{8}} \cos 45°$.
- Every probability is $1/8$.

Measuring $\mathrm{QFT}\,|x\rangle$ tells us nothing about x



$\mathrm{QFT}\,|101\rangle$ measured 2000 times with CUDA-Q

$$P(y) = \left|\frac{\omega^{xy}}{\sqrt{M}}\right|^2 = \frac{1}{M} \quad \text{for every } y$$

The histogram is **flat**, whatever x is: the information sits in the phases, which a measurement does not show.

Lesson

The QFT is useful when the input has **structure** (e.g. a period); then the output has peaks (Part 6).

The QFT can be undone: the inverse QFT

- The columns of F_M are orthonormal (by the "arrows cancel" sum), so $F_M^\dagger F_M = I$: the QFT is **unitary**.
- The **inverse QFT** just flips the sign of the phase:

$$\mathrm{QFT}^{-1}\,|y\rangle = \frac{1}{\sqrt{M}} \sum_{x=0}^{M-1} e^{-2\pi i\, xy/M}\,|x\rangle$$

- Circuit: run the QFT circuit **backwards** with all rotation angles negated.

Why it matters

Most algorithms (phase estimation, Shor) use the *inverse* QFT at the end to turn phases back into a bit string that can be measured.

The key trick: the product formula

The 2^n-term sum factorises into n single-qubit states:

Product formula

$$\mathrm{QFT}\,|x_1 x_2 \ldots x_n\rangle = \frac{1}{\sqrt{2^n}} \left(|0\rangle + e^{2\pi i\, 0.x_n}\,|1\rangle\right) \otimes \left(|0\rangle + e^{2\pi i\, 0.x_{n-1}x_n}\,|1\rangle\right) \otimes \cdots \otimes \left(|0\rangle + e^{2\pi i\, 0.x_1 x_2 \ldots x_n}\,|1\rangle\right)$$

- Every output qubit is a **phase state** $\frac{1}{\sqrt{2}}(|0\rangle + e^{i\varphi}\,|1\rangle)$.
- Its angle is a **binary fraction** built from the last bits of x.
- No entanglement in the output of a basis state — so it can be built qubit by qubit!

The next two slides prove it; you can skip the proof and trust the formula.

Proof, step 1: split y into its bits

Write $y = y_1 y_2 \dots y_n$, so $\dfrac{y}{2^n} = \dfrac{y_1}{2} + \dfrac{y_2}{4} + \cdots + \dfrac{y_n}{2^n} = \displaystyle\sum_{l=1}^{n} \frac{y_l}{2^l}$. Then

$$e^{2\pi i\, xy/2^n} = \prod_{l=1}^{n} e^{2\pi i\, x\, y_l/2^l} \qquad (e^{a+b} = e^a e^b)$$

Summing over all y means summing over every bit $y_l \in \{0,1\}$ independently, so the sum becomes a product:

$$\mathrm{QFT}\,|x\rangle = \frac{1}{\sqrt{2^n}} \sum_{y_1=0}^{1} \cdots \sum_{y_n=0}^{1} \prod_{l=1}^{n} e^{2\pi i\, xy_l/2^l}\,|y_l\rangle = \frac{1}{\sqrt{2^n}} \bigotimes_{l=1}^{n} \left(|0\rangle + e^{2\pi i\, x/2^l}\,|1\rangle \right)$$

(For $y_l = 0$ the phase is 1, for $y_l = 1$ it is $e^{2\pi i\, x/2^l}$.)

Proof, step 2: only the fractional part survives

Dividing x by 2^l moves the binary point l places to the left:

$$\frac{x}{2^l} = \underbrace{x_1 \ldots x_{n-l}}_{\text{integer}} . x_{n-l+1} \ldots x_n$$

The integer part is a whole number of turns, and $e^{2\pi i \cdot \text{integer}} = 1$, so

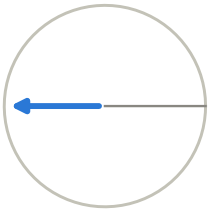
$$e^{2\pi i\, x/2^l} = e^{2\pi i\, 0.x_{n-l+1} \ldots x_n}.$$

Example $x = 5 = 101_2$

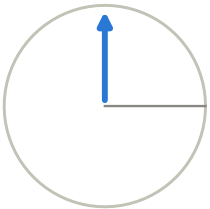
l	$x/2^l$	binary	phase factor
1	2.5	10.1_2	$e^{2\pi i\, 0.1_2} = e^{i\pi} = -1$
2	1.25	1.01_2	$e^{2\pi i\, 0.01_2} = e^{i\pi/2} = i$
3	0.625	0.101_2	$e^{2\pi i\, 0.101_2} = e^{i5\pi/4}$

What the output qubits look like for $x = 5$

y_1: $0.x_3 = 0.1_2$
phase $2\pi \times 0.5$



y_2: $0.x_2x_3 = 0.01_2$
phase $2\pi \times 0.25$



y_3: $0.x_1x_2x_3 = 0.101_2$
phase $2\pi \times 0.625$



$$\mathrm{QFT}\,|101\rangle = \tfrac{1}{\sqrt{8}}\big(\,|0\rangle - |1\rangle\,\big) \otimes \big(\,|0\rangle + i\,|1\rangle\,\big) \otimes \big(\,|0\rangle + e^{i5\pi/4}\,|1\rangle\,\big)$$

Each output qubit is a clock hand on the "equator". Multiplying out the brackets gives back the 8 amplitudes $\omega^{5y}/\sqrt{8}$ from the CUDA-Q table.

Ingredient 1: the Hadamard writes the first bit

Recall $H|b\rangle = \frac{1}{\sqrt{2}}(|0\rangle + (-1)^b|1\rangle)$ and $(-1)^b = e^{i\pi b} = e^{2\pi i\, b/2} = e^{2\pi i\, 0.b}$:

$$H|x_j\rangle = \frac{1}{\sqrt{2}}\left(|0\rangle + e^{2\pi i\, 0.x_j}|1\rangle\right)$$

- After the Hadamard, the qubit is a phase state whose angle is $0.x_j$ in binary: 0° if $x_j = 0$, 180° if $x_j = 1$.
- We still need to add the *following* bits: $0.x_j \to 0.x_j x_{j+1} \ldots x_n$.

Ingredient 2: controlled rotations add the next bits

A controlled-R_k with control x_m multiplies the $|1\rangle$ part of the target by $e^{2\pi i/2^k}$ *only if* $x_m = 1$, i.e. it adds $x_m/2^k$ to the phase fraction. Since $0.x_j + x_{j+1}/4 = 0.x_j x_{j+1}$:

$$e^{2\pi i\, 0.x_j}$$

$$\xrightarrow{R_2,\ \text{control } x_{j+1}} e^{2\pi i\, 0.x_j x_{j+1}}$$

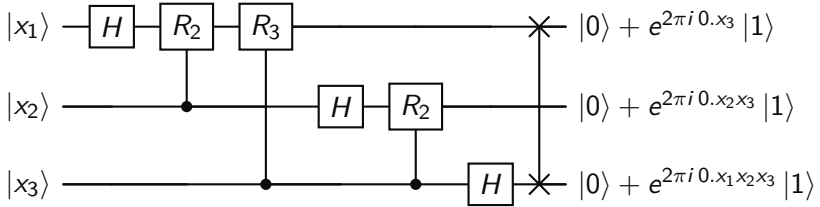
$$\xrightarrow{R_3,\ \text{control } x_{j+2}} e^{2\pi i\, 0.x_j x_{j+1} x_{j+2}} \ \rightarrow\ \ldots$$

Recipe for qubit j

1. Hadamard on qubit j
2. R_2 controlled by qubit $j{+}1$
3. R_3 controlled by qubit $j{+}2$
4. $\ldots$ up to R_{n-j+1} controlled by qubit n

Result: $\frac{1}{\sqrt{2}}(|0\rangle + e^{2\pi i\, 0.x_j \ldots x_n}\,|1\rangle)$

The 3-qubit QFT circuit



- Top wire: H, then R_2 (control x_2) and R_3 (control x_3) $\Rightarrow$ phase $0.x_1x_2x_3$.
- Middle wire: H, then R_2 (control x_3) $\Rightarrow$ phase $0.x_2x_3$. Bottom wire: $H \Rightarrow 0.x_3$.
- The SWAP puts the qubits in the order of the product formula (normalisation $1/\sqrt{8}$ omitted).

Walk-through: $|101\rangle$ through the circuit

After its Hadamard each wire is a phase state $\frac{1}{\sqrt{2}}(\,|0\rangle + e^{2\pi i f}\,|1\rangle\,)$; the table tracks the binary fraction f.

after gate	wire 1	wire 2	wire 3
start	$\|1\rangle$	$\|0\rangle$	$\|1\rangle$
H on wire 1	$f = 0.1_2$	$\|0\rangle$	$\|1\rangle$
R_2, control wire 2 $(= 0)$	$f = 0.10_2$ (no change)	$\|0\rangle$	$\|1\rangle$
R_3, control wire 3 $(= 1)$	$f = 0.101_2$	$\|0\rangle$	$\|1\rangle$
H on wire 2	·	$f = 0.0_2$	$\|1\rangle$
R_2, control wire 3 $(= 1)$	·	$f = 0.01_2$	$\|1\rangle$
H on wire 3	·	·	$f = 0.1_2$
SWAP wires 1 and 3	0.1_2: phase -1	0.01_2: phase i	0.101_2: phase $e^{i5\pi/4}$

This is exactly the product formula for $x = 5$. Wires 2 and 3 are used as **controls before** they receive their own Hadamard — that is why the order of the gates matters.

Why the SWAPs at the end?

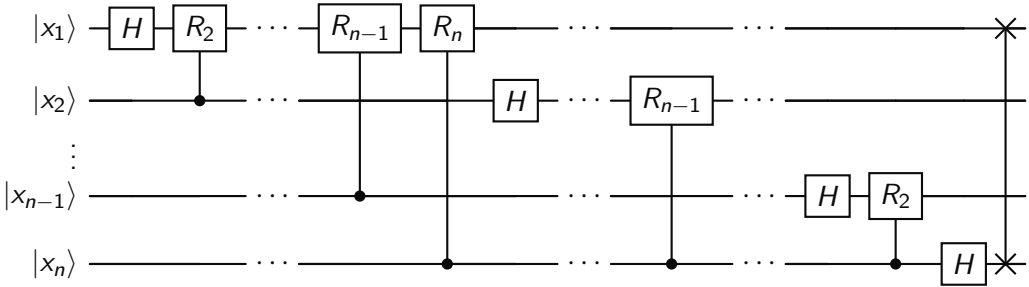
- The circuit leaves qubit 1 with the phase $0.x_1x_2\ldots x_n$ — but the product formula wants that phase on the **last** qubit.
- The outputs come out in **reversed order**.
- Fix: SWAP qubit $1 \leftrightarrow n$, $2 \leftrightarrow n-1$, ...: $\lfloor n/2 \rfloor$ swaps.
- Many algorithms skip the swaps and simply read the bits in reverse order (it is free).

A SWAP is 3 CNOTs

SWAP exchanges two qubits: $|ab\rangle \mapsto |ba\rangle$.

It can be built from three CNOTs with alternating directions.

The general n-qubit circuit



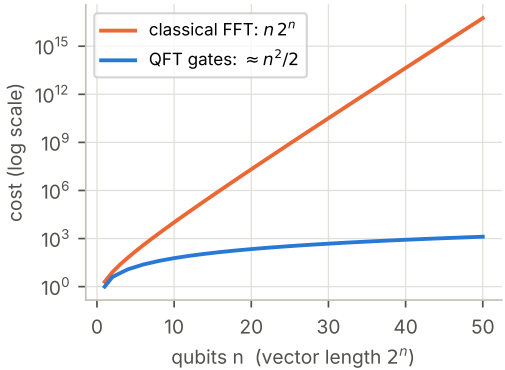
Qubit j gets one Hadamard and $n-j$ controlled rotations $R_2, \ldots, R_{n-j+1}$; then the order is reversed with SWAPs (only the outer pair is drawn).

Counting gates: why the QFT is fast

- Hadamards: n
- controlled rotations:
  $(n-1)+(n-2)+\cdots+1 = \dfrac{n(n-1)}{2}$
- SWAPs: $\lfloor n/2 \rfloor$

Total

$\approx n^2/2$ gates for 2^n amplitudes. $n = 15$: $15 + 105 + 7 = 127$ gates;
$n = 30$: 480 gates.



Exponential (FFT) vs. quadratic (QFT) growth

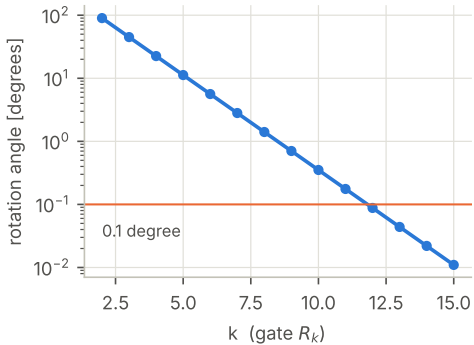
The approximate QFT

R_k rotates by $360°/2^k$ — tiny for large k:

- R_{10}: $0.35°$, R_{15}: $0.011°$
- Real gates have errors larger than that anyway.

Approximate QFT

Keep only R_k with $k \leq m$ ($m \approx \log_2 n + 2$). The result is almost identical, but needs only about $n\,m$ gates, i.e. $O(n \log n)$.



Periodic states: the math

A state with period r (dividing M) and offset o uses $K = M/r$ basis states:

$$|\psi\rangle = \frac{1}{\sqrt{K}} \sum_{j=0}^{K-1} |j\, r + o\rangle$$

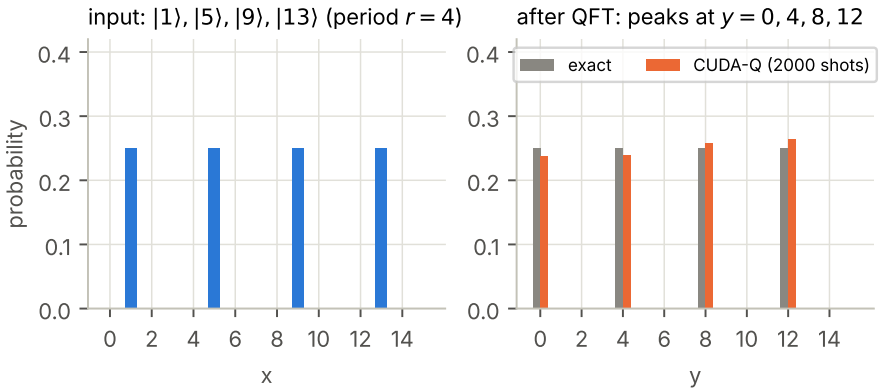
Apply the QFT and collect the amplitude of $|y\rangle$:

$$\beta_y = \frac{1}{\sqrt{MK}} \sum_{j=0}^{K-1} \omega^{(jr+o)y} = \frac{\omega^{oy}}{\sqrt{MK}} \underbrace{\sum_{j=0}^{K-1} e^{2\pi i\, j\, ry/M}}_{=K \text{ if } ry/M \in \mathbb{Z}\text{, else } 0}$$

Result

Only $y = 0, \frac{M}{r}, \frac{2M}{r}, \ldots$ survive, each with probability $1/r$ — the same pattern as the classical DFT example in Part 1.

Example: 4 qubits, period 4, offset 1



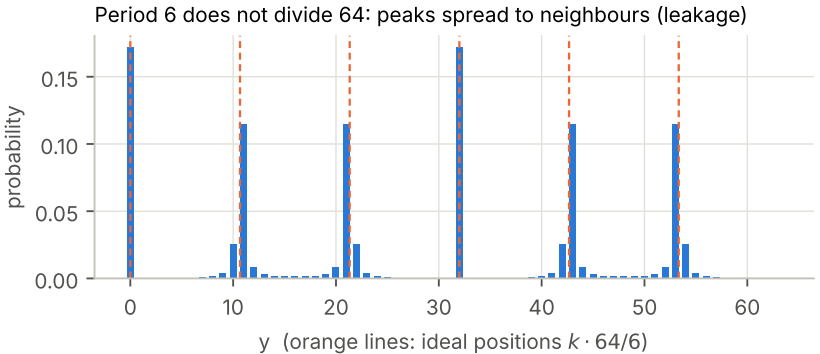
$M = 16$, $r = 4$: input $\frac{1}{2}(|1\rangle + |5\rangle + |9\rangle + |13\rangle)$. Prediction: peaks at multiples of $16/4 = 4$, each with probability $1/4$. The CUDA-Q simulation (2000 shots) agrees; the small differences are shot noise ($\approx 1/\sqrt{2000}$).

The offset hides in the phases

y	β_y (CUDA-Q)	$\lvert\beta_y\rvert^2$	angle
0	$+0.500 + 0.000i$	0.250	0°
4	$+0.000 + 0.500i$	0.250	90°
8	$-0.500 + 0.000i$	0.250	180°
12	$+0.000 - 0.500i$	0.250	270°

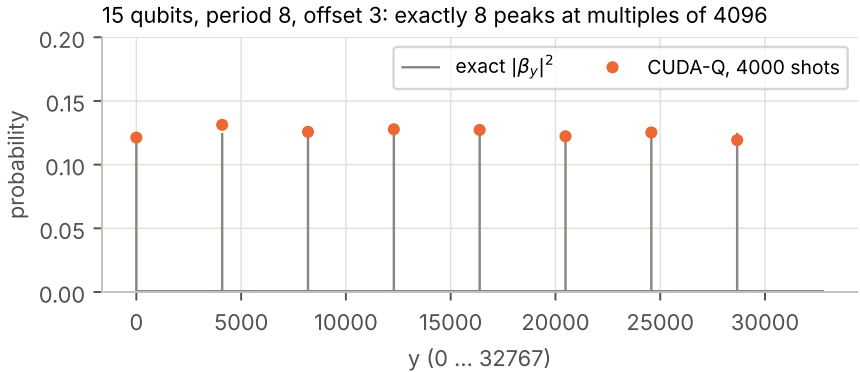
- Theory: $\beta_y = \omega^{oy}/\sqrt{r}$ with $o = 1$: angles $360^\circ \cdot y/16$.
- Changing the offset o only rotates these phases — the **probabilities do not change**.
- This "shift invariance" is why the QFT finds the period no matter where the pattern starts.

When the period does not divide M: leakage



$n = 6$ ($M = 64$), period $r = 6$: $64/6 \approx 10.67$ is not an integer. The arrows no longer cancel perfectly: the peaks sit **near** $k \cdot 64/6$ and spread to neighbouring y. More qubits make them sharper; Shor's algorithm handles this with **continued fractions** (two slides ahead).

A bigger simulation: 15 qubits with CUDA-Q



$M = 32\,768$ amplitudes, period 8, offset 3 (the default example of `QFT.ipynb`). CUDA-Q's state agrees with NumPy's FFT with fidelity 1.000000000000 (largest amplitude error 4.6e-16). With 4000 shots we only ever see the 8 peaks — the other 32 760 outcomes have probability zero.

From a measured peak to the period

A measurement gives one y with $\frac{y}{M} \approx \frac{k}{r}$ for some unknown integer k.

Example ($M = 64$, true period $r = 6$)

We measure $y = 11$: $11/64 = 0.171875$.
Best fraction with a small denominator: $1/6 = 0.1667 \Rightarrow$ guess $r = 6$. ✓

We measure $y = 21$: $21/64 = 0.328 \approx 1/3 \Rightarrow r = 3$ is only a divisor of 6 — measure again.

- The "best fraction" comes from the **continued-fraction expansion** (in Python: `Fraction(11, 64).limit_denominator(10)`).
- A few shots are enough to find r with high probability.
- In **Shor's algorithm**, r is the period of $a^x \bmod N$ and it reveals the factors of N.

Where the QFT is used

Quantum phase estimation

Measures the eigenvalue $e^{2\pi i\varphi}$ of a unitary: the core of quantum chemistry (molecular energies) and of HHL (linear systems).

Shor's factoring algorithm

Finds the period of $a^x \bmod N$; breaks RSA on a large fault-tolerant quantum computer.

Quantum arithmetic

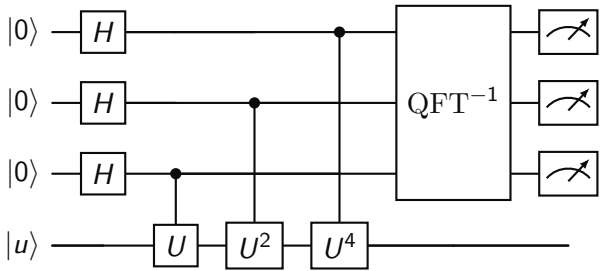
Adding numbers "in Fourier space" needs only phase rotations (Draper adder).

Hidden structure

Hidden subgroup problems, period finding in general.

The notebooks `Shor.ipynb` and `HHL.ipynb` in this repository both contain an inverse QFT.

Inside phase estimation: the inverse QFT at work



- $U|u\rangle = e^{2\pi i\varphi}|u\rangle$. The controlled powers U^{2^j} write the phases $e^{2\pi i 2^j\varphi}$ onto the top qubits — exactly the pattern of the product formula.
- The inverse QFT turns this into the bit string $\varphi \approx 0.\varphi_1\varphi_2\varphi_3$, which we measure.

Limits: what the QFT does *not* do

- **No direct read-out**: the 2^n results are amplitudes. One shot gives one bit string; estimating all probabilities needs $\sim 2^n$ shots, and phases need even more. Useful algorithms only need a few peaks.
- **Input preparation**: loading an arbitrary list of 2^n numbers into a quantum state is itself expensive. The speed-up only pays off when the algorithm prepares the input (as in Shor or phase estimation).
- **Precision and noise**: tiny rotation angles and $\approx n^2/2$ gates make the QFT sensitive to gate errors — see `Noise_Simulation.ipynb`; the approximate QFT helps.
- **Simulation**: a classical simulator stores all 2^n amplitudes, so simulations stop at $\approx$ 30–45 qubits even on large GPU clusters.

The QFT in CUDA-Q (from `QFT.ipynb`)

```
import cudaq, numpy as np

@cudaq.kernel
def qft(qubits: cudaq.qview):
    n = qubits.size()
    for j in range(n):
        h(qubits[j]) # Hadamard
        for k in range(j + 1, n): # R_(k-j+1)
            r1.ctrl(np.pi / 2**(k - j),
                    qubits[k], qubits[j])
    for j in range(n // 2): # reverse order
        swap(qubits[j], qubits[n - 1 - j])

@cudaq.kernel
def demo():
    q = cudaq.qvector(3)
    x(q[0]); x(q[2]) # |101> = |5>
    qft(q)

print(cudaq.draw(demo))
state = cudaq.get_state(demo)
```

- `r1(angle)` $= \mathrm{diag}(1, e^{i\cdot\mathrm{angle}})$, so `r1`$(\pi/2^{k-j}) = R_{k-j+1}$.
- `.ctrl(c, t)` makes it controlled.
- `cudaq.get_state` returns the 2^n amplitudes; `cudaq.sample` measures.
- Note: `get_state` lists amplitudes with qubit 0 as the *least* significant bit; the notebook converts this for you.

Try it yourself: `QFT.ipynb`

1. Install: `pip install cudaq numpy scipy matplotlib psutil`
2. Open `QFT.ipynb` and run all cells.
3. Change only Section 3:
   - `N_QUBITS` (default 15)
   - `INITIAL_STATE`: `"periodic"`, `"basis"`, `"random"`, `"custom"`
   - `PERIOD`, `OFFSET`
   - `BACKEND`: `"cpu"` or `"gpu"`

Experiments

- `PERIOD = 6`: see leakage
- `INITIAL_STATE = "basis"`: a flat histogram, information only in the phases
- `N_QUBITS = 25` on a GPU: compare the execution time with the CPU

The notebook compares every result with NumPy's FFT and reports time and memory.

Exercises

1. Compute $\mathrm{QFT}\,|11\rangle$ for two qubits ($x = 3$).
2. What is $\mathrm{QFT}\,|000\rangle$? What do you measure?
3. For $x = 3 = 011_2$ on three qubits, give the phase of every output qubit (product formula).
4. How many gates does the QFT on $n = 10$ qubits need (including SWAPs)?
5. A 5-qubit state has period 8. Where are the peaks after the QFT, and how likely is each one?
6. Why can't we read out all 2^n Fourier coefficients efficiently?
7. Bonus: show that F_4 applied twice maps $|x\rangle$ to $|-x \bmod 4\rangle$.

Answers

1. Column $x = 3$ of F_4: $\frac{1}{2}\big(|00\rangle - i\,|01\rangle - |10\rangle + i\,|11\rangle\big)$.
2. $\frac{1}{\sqrt{8}}\sum_y |y\rangle$ — all phases are 1 ($x = 0$). Every outcome has probability $1/8$.
3. $x_1x_2x_3 = 011$: $0.x_3 = 0.1_2 \to 180°$; $0.x_2x_3 = 0.11_2 = 3/4 \to 270°$; $0.x_1x_2x_3 = 0.011_2 = 3/8 \to 135°$.
4. $10 + 45 + 5 = 60$ gates.
5. $M = 32$, $M/r = 4$: peaks at $y = 0, 4, 8, \ldots, 28$; 8 peaks with probability $1/8$ each.
6. Each shot returns one bit string; estimating 2^n probabilities needs about 2^n shots (and the phases are not visible in a measurement at all).
7. $F_4^2\,|x\rangle = \frac{1}{4}\sum_{y,z} i^{xy}\,i^{yz}\,|z\rangle = \sum_z \big(\frac{1}{4}\sum_y i^{y(x+z)}\big)\,|z\rangle$; the inner sum is 1 only if $x + z \equiv 0 \pmod 4$, so the result is $|-x \bmod 4\rangle$.

Summary

- The Fourier transform changes the view from **time** to **frequency**; periods become peaks.
- The QFT applies the DFT to the 2^n **amplitudes** of an n-qubit state: $|x\rangle \mapsto \frac{1}{\sqrt{M}} \sum_y \omega^{xy} |y\rangle$.
- The **product formula** turns it into one phase state per qubit, with angles given by **binary fractions** of x.
- Circuit: **Hadamards + controlled rotations + SWAPs**, only $\approx n^2/2$ gates.
- The output is in the amplitudes: useful when we only need a few **peaks**, e.g. a period.

Further reading

- M. A. Nielsen, I. L. Chuang, *Quantum Computation and Quantum Information*, Chapter 5
- D. Coppersmith, *An approximate Fourier transform useful in quantum factoring* (1994)
- NVIDIA CUDA-Q documentation
- This repository: `QFT.ipynb`, `Shor.ipynb`, `HHL.ipynb`